

Reflective Journal — Lab 04

SingleEpoch | ITAI 1371 Module 04

1. What did EDA change about how you see the Titanic features?

Honestly, before I ran the plots I was kind of just staring at column names. Survived, Pclass, Sex, Age, Fare, Embarked. I could pull numbers out of `.describe()` like the survival rate being around 38%, average age around 30, fares going from basically nothing up to like \$512. Cool facts, but they did not really land for me yet.

Once I actually looked at the plots, that changed.

Sex jumped out first. It was not just another field. Women clearly made it more often than men. Same thing with class. First class passengers did way better, and third class got hit hard. Age was the one that surprised me the most though. On paper it is "about 30 years old," but the FacetGrid showed this spike of young kids on the survived side that a single average just completely covers up.

My two experiments kind of locked that in. For Embarked, Cherbourg was the only port where more people lived than died, and Southampton looked the worst. For Fare, people who survived tended to pay more, and that wild \$512 ticket showed up on the survived side.

So yeah, the big takeaway for me is that tables give you the averages, but the plots are what actually show you the story. Who had better odds, where the weird bumps are, stuff a mean or a count is never going to tell you on its own.

2. Where did data quality force a decision?

When I ran `.info()`, the missing stuff was right there. Out of 891 passengers, Cabin was empty for most of them, like around 687 people. Age was missing for about 177. Embarked only had two blanks. Everything else I plotted was pretty much filled in.

Even though this lab was just EDA, that still made me stop and think about what I would do later if I had to clean this for a model.

Cabin is basically too empty to treat like a normal category. I would probably drop it, or turn it into something simpler like whether they had a cabin at all, or maybe just the deck letter. Trying to pretend all those cabin codes are usable as is does not make sense to me.

Age is trickier because you can still use it, but you are missing a chunk of people. The age plot only shows passengers who actually have an age written down, so that kid spike I noticed is already incomplete. Later on I would not just delete those 177 rows and move on. I would either fill them in somehow, maybe with a median by Sex and Pclass, or keep a note that age was missing. For this lab leaving the blanks alone was fine.

Embarked was the easy one. Two missing values. Fill them with S since that is the most common port, or just drop those two rows. Not really comparable to the Age or Cabin problem.

Fare did not have missing values, but it did have that \$512 outlier. So if I took this further I would have to decide whether to leave it, cap it, or do a log transform so one ticket does not throw everything off.

Overall the missing data did not stop me from finishing Lab 04, but it did make it obvious that not every column is ready for modeling just because it is in the table.

3. RCA: the string literal bug (cells 2 and 4)

I am using this one instead of the missing Age and Cabin stuff, because this is what actually stopped me when I tried to run the notebook.

Symptom: I ran cell 2 and immediately got a `SyntaxError` about an unterminated string literal on the print that was supposed to show the Basic Info banner.

Cause: Looking at the notebook JSON, that print string had gotten split across two pieces in the cell source. The opening quote was on one entry, and the rest of the text was on the next. When the editor glued them together it put a real newline in the middle of the string, so Python freaked out. Cell 4 had the same problem on the Key Insights print. The other code cells were fine.

Fix: I just merged those split pieces back into one normal line in cells 2 and 4. Pretty small fix. Nothing fancy.

Why: This was already sitting in the instructor notebook. I did not introduce it. Putting the string back together was the simplest correct thing to do. Rewriting the whole cell or skipping `.info()` and `.describe()` would have just avoided a step the lab expects you to run.

What I took from it is that a notebook can look totally fine on the screen and still be broken underneath. So now I am more careful about actually running each code cell before I trust anything that comes after it.

4. How this connects to Module 3 (without rehashing Wine)

I am not dragging Wine results into this writeup. What actually carried over from Module 3 is how I work through an assignment.

Same basic process: read the spec, figure out the cells, change one thing at a time, check that the output looks right, run the local gate and CI, and only do Restart and Run All at the end so I know the whole notebook is clean.

What is different here is the assignment itself. This time it is Titanic, not Wine, and the definition of done is exploring the data, doing two short plot experiments, and writing the journals. No model training. No learning curves.

So Module 3 is where I learned the workflow. Lab 04 was me using that same workflow on a new dataset: look at the features, notice where the data is messy, write up the RCA when something breaks, and stop once this Canvas definition of done is actually met.